Lectures Machine Learning

Machine Learning (Vrije Universiteit Amsterdam)

Machine Learning, Lecture 1: Introduction

- Machine learning (ML) is the application of the principle of learning to a computer: provide the computer with a set of examples, than try to figure out a program that recognized the regularities in the examples and ignores irrelevant details.
- Suitable ML problems:
  - Can't be solved explicitly.
  - Approximations are good enough.
  - Limited reliability, predictability, and interpretability are fine.
  - Plenty of examples to learn from available.
- Definition ML: ML provides systems the ability to automatically learn and improve from experience without being explicitly programmed.
- ML is often built on abstract tasks; abstract tasks can be split up into two categories: supervised and unsupervised.
  - Supervised: explicit examples of input and output, learn to predict the output for an unseen input
  - Unsupervised: only input is provided, find any pattern that explains something about the data
- Supervised learning tasks:
  - Classification: assigning a class to each example
  - Regression: assign a number to each example
- ML is a subfield of Artificial Intelligence (AI), so all ML: is AI but not all AI is ML
- In the same way, ML is a form of Data Science (DS), so all ML is data science but not all DS is ML
- ML vs Data Mining: both analyze data, ML with the goal to produce a model of the data, while DM aims to produce intelligence about the data. (result ML = software, result DS = knowledge)
- ML vs Statistics: ML aims for a model that works but doesn't necessarily have to be true. Statistics always aim for the truth.
- Classification:
  - The data we provide our system with consists of examples, called instances, of the things we are trying to learn something about.
  - We than make a series of measurements about each instance, the things we measure are called the features of the instance.
  - Finally, we have the target value: the thing we are trying to learn.
  - This dataset is fed to a learning algorithm. This can be anything, but it has to produce a classifier. A classifier is a small "machine" that makes the required class predictions.
- Sometimes, before we can apply a classification algorithm, we need to translate a problem into an abstract task before we can apply a classification algorithm.
- There are different types of linear classifiers: a linear classifier, a decision tree classifier and a nearest neighbors classifier.
- A loss function tells us how much we like the model for the current data. The lower the outcome, the better the model. A loss function uses the model as its argument and the data as a constant.
- Another type of classifier: a decision tree. A decision tree studies one feature in isolation at every node. Often, decision trees are "grown" by adding nodes from the root until a particular criterion is reached.

- A last type of classifier is a lazy type of classifier: the k-nearest neighbors classifier. This classifier doesn't learn anything, it simply remembers the data. It will look at the k-amount of closest data points in comparison to the new data and assign the class that is most frequent in the nearest data points. (K is called a hyperparameter: you have to choose it yourself before you use the algorithm.
- Basic recipe for machine learning: take a problem, translate the problem to an abstract task → choose instances and features → choose a model class → search the model space for a model that solves the problem well.
  In code:

```
# choose a model: decision trees
from sklearn.tree import DecisionTreeClassifier

x_train = … # matrix of features
y_train = … # target class labels

tree = DecisionTreeClassifier()
tree.fit(x_train, y_train) # search for model

# classify some new data
x_new, y_new = … # features and labels
y_predicted = tree.predict()
# see how well we do
print(accuracy_score(y_predicted, y_test))
```

We have now looked at classification, but there are also other forms of the supervised abstract task such as regression.

- Regression works exactly the same as classification, except we are predicting a number instead of a class
- Loss function for regression = mean-squared-errors loss = MSE (a loss function maps a model to a number that expresses how well it fits the data → the smaller the loss, the better). The MSE is a common choice for a regression. It uses the difference between the model prediction and the target value from our data for each instance, we call this the residual. We square and sum all residuals → the lower the outcome, the better the model fits our data.
- In a linear regression, you choose the line through the data with the lowest MSE for this data.
- We can also use the decision tree principle to perform a regression, this will result in a regression tree. This can be done by segmenting the feature space into blocks, using a tree. But this time, instead of assigning a class to each, we will assign a number.
- There also exists a regression equivalent to the kNN classifier: the kNN regression.

Next to the supervised abstract tasks, we can also look into to unsupervised tasks. In that case, we don't have output, only input. The task for the model is to find any useful structure in the data.

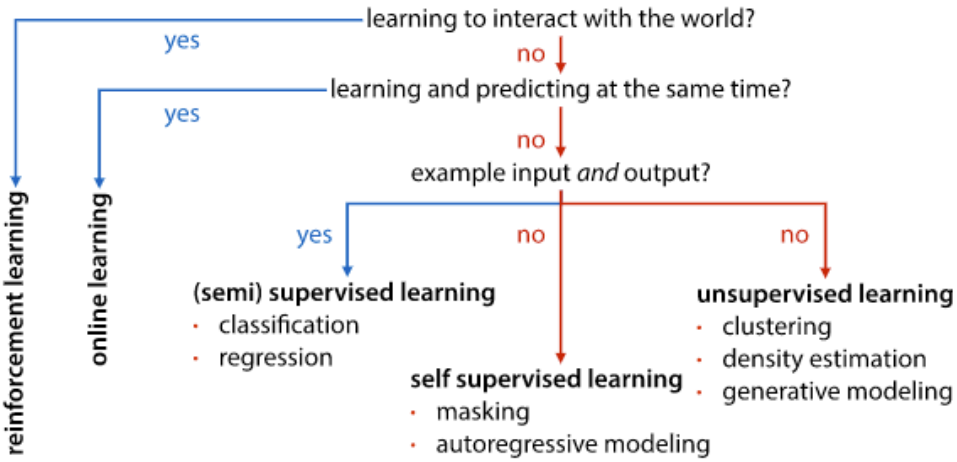
- In the case of clustering, we ask the learner to split the instances into a number of clusters. The number of clusters is usually given beforehand by the user. This method looks a lot like classification but note that there are no example classes provided by the data.
- One example of a clustering algorithm is k-means. We start by choosing three random points in the feature space, called the means. Each of these represents one

of our clusters. We then assign each point to the cluster corresponding to the mean its closest to. Since all means were randomly chosen, this does not yet correspond to a very meaningful clustering of data. After this step, the means are re-computed. Each new mean is the mean of all the points that now belong to its cluster. This procedure (re-assigning clusters and re-computing the means) is iterated until the means stop moving from one iteration to the next.

- Another way to approach the abstract task is by doing a density estimation. In a density estimation, we want to learn how likely new data is. For each instance, a number is predicted, and that number expresses how likely the model thinks the given instance is. This number should behave as a probability density and therefore can't be negative, and all the numbers the model produces should sum or integrate to one.
- With highly complex data, it is often easier to sample from a probability distribution that it is to get a probability density estimate. Building a model from which you van sample new examples is called generative modelling.

Apart from supervised and unsupervised tasks, we also have semi-supervised learning. Unlabeled data is broadly available, while labeled data is expensive to acquire. In such cases, semi-supervised learning can be useful: this involves learning from a small, labeled set and a large amount of unlabeled data.

Brief overview:



Machine Learning, Lecture 2: Linear Models and Search

- In ML each instance is described by n features. Each feature can be represented by a vector. A feature has two parameters in a standard linear regression model: the weight and the bias. The weight is also sometimes called the slope, and the bias the intercept. The bias determines where the line crosses the vertical axis, the weight determines how much the line rises if we move one step to the right.
- To minimize the loss, we need to search the for the lowest point in the loss landscape. The mathematical name for this operation is optimization: we are trying to find the input for which a particular function (the loss) is at its optimum. If that is not possible, we want to find the lowest value possible.

- ML vs optimization: optimization is concerned with finding the absolute minimum or maximum, ML is concerned with finding the lowest loss that generalizes → ML wants to minimize the loss on the test data, while only seeing the training data.
- Random search: start with a random point in the model space → pick a point close to the randomly chosen point, if the loss goes up → move back to the previous point. If the loss goes down → stay in the new point. Repeat process. This approach works well enough as long as our problem is a convex problem. This means that if a line is drawn between two points on the surface, it lies entirely above the surface.
- What happens if the loss surface is not convex? This will disturb the iterative process, as it will get stuck as soon as a local minimum has been found → the results will not be reliable. A way to escape from local minima is by simulated annealing: we will sometimes allow the algorithm to travel uphill.
- The space of linear models is continuous: between every two models, there is always another model, not matter how close they are together. The alternative is a discrete model space. Another thing you can do is just to run a random search a couple of times independently: parallel search.
- To make parallel search even more useful, we can introduce some form of communication or synchronization between the searches happening in parallel. If we see the parallel searches as a population of agents that occasionally communicate in some way, we can guide the search a lot more. We call this population methods, and there are multiple methods:
    - Evolutionary algorithms:
        - Genetic algorithms
        - Evolutionary strategies
    - Particle swarm optimization
    - Ant colony optimization

  Population methods are very powerful, but computing the loss for so many different models is often expensive. They can also come with a lot of different parameters to control the search, each of which needs to be tuned carefully. Population methods are powerful, easy to parallelize, slow/expensive for complex models, difficult to tune.
- To escape local minima: add randomness, add multiple models. To converge faster: combine known, good models (population methods).

Gradient descent: the basis of the gradient descent algorithm is the derivation of a continuous model space with a smooth loss function. Outline: using calculus, we can find the direction in which the loss drops most quickly. This direction is the opposite of the gradient. Gradient descent takes small steps in this direction in order to find the minimum of a function.

- The tangent hyperplane of a function f approximates the function f locally
- The gradient gives the slope of the tangent hyperplane
- The vector expressing the slope of a hyperplane is also the direction of the steepest ascent on that hyperplane, the opposite vector is the steepest descent

→ Conclusion: to move to a lower point of f, we can compute the gradient and take a small step in the opposite direction.

Gradient descent algorithm: starting from a random point p, we compute the gradient at point p. We subtract it from the current choice and iterate the process. We can iterate until the loss gets low enough, or until the gradient gets close enough to the zero vector, which implies we have reached a local minimum.
Gradient descent only works for continuous model spaces, with smooth loss functions, for which we can work out the gradient does not escape local minima. If we can use gradient descent, it is very fast, low memory and very accurate. Gradient descent is the backbone of 99% of modern machine learning.

Classification: how do we define a linear classifier? → a classifier whose decision boundary is always a line (or hyperplane) in the feature space.

- To define a linear decision boundary, we take the same functional form we used for the linear regression: some weight vector w, and a bias b. The way we define the decision boundary is a little different than the way we defined the regression line.

Loss functions serve two functions:

- To express what quantity we want to maximize in our search for a good model
- To provide a smooth loss surface, so that we can find a path from a bad model to a good one

→ therefore, it is common not to use the error as a loss function, even when it's the thing we are actually interested in minimizing. Instead, we will replace it by a loss function that has its minimum at the same model, but that provides a smooth, differentiable loss surface.

Three common loss functions for classifications: least squares loss, log loss/cross entropy/SVM loss. Least squares loss is just an application of MSE loss to classification. Usually, this method is not very good, but it will give you an idea of what a classification loss might look like. The least squares classifier essentially turns the classification problem into a regression problem: it assigns points in one class +1 and points in another class -1, we then use basic MSE loss that we saw earlier, to train a regression model to predict these numeric values. Performing gradient descent with this loss function will result in a line that minimizes the residuals. Hopefully, the points are far enough apart that the decision boundary will separate the two classes. This will probably result in very large residuals: that is because the linear model is not appropriate. With some luck, the best fitting line will be positive for the +1 class and negative for the -1 class. If so: the classifier makes the right predictions.

Machine Learning, Lecture 3: Model evaluation
Basic recipe for machine learning:

- Abstract part of your problem to a standard task, such as: classification, regression, clustering, density estimation, generative modelling, online learning, reinforcement learning, or structured output learning.
- Choose your instances and their features. (for supervised learning: choose a target)
- Choose your model class: linear models, decision tree, kNN.
- Search for a good model. Usually, a model comes with its own search method. Sometimes multiple options are available.

Binary classification = two class classification. The main metric of performance for a binary classification is the proportion of misclassified examples. This is called the error. The proportion of correctly classified examples is called the accuracy.

Never judge the performance of a classifier by the results achieved on the training data. We train our classifiers on the training data and test on the test data (The test data is part of the data we take from the complete dataset. The test data should consist of at least 500 examples, but 10 000 or more is ideal)

Do not do multiple tests on the test data, as this will result in overfitting (since you are testing your data again and again on the same datapoints. To avoid this:

- Split your data into train and test data (in long term projects you might want to split up into multiple sets of test data, so you can iterate the process of testing.
- Choose your model, hyperparameters, etc. only using the training set. (only use your test data for last minute testing)
- State your hypothesis.
- Test the hypothesis once, on the test data.
- Reusing the test data means that you will pick the wrong model and it will result in a much lower error than is realistic.

In order to prevent using the test data multiple times, the usual approach is as follows:

- During model and hyperparameter selection we want to train on the training data and test on validation data. The validation data is taken from the training set.
- For the final run, we want to train on the training and validation data, and test on the test data.

Cross validation:
Sometimes when you have split off a test and a validation set, you might be left with very little training data. If this is the case, we can make better use of the training data by performing cross validation: split the data into 5 folds and for each specific choice of hyperparameters we want to test we do five runs: each with one of the folds as validation data. Afterwards, we average the scores of these runs. This can take a lot of time as we need to train five times as many classifiers.

Temporal data: sometimes data can have a meaningful temporal ordering of the instances. For temporal data, we cannot train on samples that are in the future → you can't sample randomly. Sometimes data does have a timestamp, but there is no meaningful information in the order. In this case we can sample the test randomly.

Evaluation is essentially simulation of production. Validation is a simulation of an evaluation.

Which hyperparameters to try:

- Trial and error (intuition) (most common)
- Grid search (define a set of values per hyperparameter, try all combinations.
- Random search

Statistical analysis for reported results:

- Standard deviation: measure of spread, variance
- Standard error/confidence interval: measure of confidence

Evaluation metrics:
- MSE = mean squared errors
- RMSE = real mean squared errors
- (RMSE is easier to interpreted as it has the same units as the original output data)
- Bias = difference between true MSE and the measured MSE. When bias is high, the model does not fit the generating distribution. Poor assumptions + poor capacity → underfitting.
- High variance: high model capacity, sensitivity to random fluctuations → overfitting.
- Reducing bias → increase model capacity and increase features.
- Reducing variance → reduce model capacity, add regularization, reduce tree depth.
- K-NN regression: increase k to increase bias and decrease variance.

Cost imbalance: do the misclassifications balance out and is the classification worth the risk of misclassifying.
Precision and recall are two metrics that express a tradeoff between two types of mistakes:
- Precision: what proportion of the returned positives are actually positive
- Recall: what proportion of the existing positives did we find

- TPR = true positive rate = proportion of positives we got right → the higher the better
- FPR = false positive rate = proportion of positives that were not actually positive → the lower the better
- ROC = receiver-operating characteristic = TPR/FPR space
- We can make an ROC- curve. The area under the ROC-curve, the AUC, is a good indication of the quality of the classifier. The bigger the area the more useful the classifiers we can achieve.
- To interpret the AUC, you should know not just what classifier was used, but how it was made into a collection of classifiers. You should also know whether it's the area under an ROC curve, or a precision/recall curve.

Which dataset:
- Test accuracy: final test of model performance
- Validation accuracy: to choose hyperparameters
- Training accuracy: the difference between validation and training error will tell you if your model is overfitting or underfitting

Machine Learning, Lecture 4: Probabilistic models
We can use different criteria to decide what model we want to pick. The most common criterion is that we should guess the model for which the probability of seeing the data that we saw is highest. This is called the maximum likelihood principle.

Maximum likelihood principle: under the maximum likelihood principle, picking a model becomes an optimization problem.
Log-likelihood: what we maximize to fit a probability model.

Loss: what we minimize to fit a machine learning model.

In practice, we very often use the log-likelihood instead of the likelihood. The logarithm is a monotonic function (always gets bigger when the input gets bigger) so the likelihood and the log-likelihood have their maxima in the same place, but the log-likelihood is often easier to manipulate symbolically.

- Generative classifier: $p(Y|X) \propto p(X|Y)p(Y)$ → apply Bayes. A generative classifier focuses on learning a distribution on the feature space given the class $p(X=s|Y)$. This distribution is then combined with Bayes' rule to get the probability over the classes, conditioned on the data.
- Discriminative classifier: learn a function for $p(Y|X)$ directly with X as unput and class probabilities as output. It functions as a kind of regression, mapping x to a vector of class probabilities.

Bayes classifiers:

- Bayes optimal classifier: marginalizes over all classifiers in a model class. Probably optimal (given certain assumptions). Usually too expensive to compute. (= very accurate, but not very practical)
- Bayes classifier: learns single distribution $P(X|Y)$. Reasonable approach for low-dimensional data.
- Naïve Bayes classifier: assumes conditionally independent features. Simple, cheap and effective for high-dimensional data.

Bayes classifier:

- Choose probability distribution M (e.g. MVN = multivariate normal distributions)
- Fit Mpos to all positive points: $p(X=x|pos)=Mpos(x)$
- Fit Mneg to all negative points: $p(X=x|neg)=Mneg(x)$
- Estimate P(Y) from the class frequencies in the training data, or use domain-specific information.
- Compute terms $tpos=Mpos(x)p(pos)$ and $tneg=Mneg(x)p(neg)$
- Compute class probabilities $p(pos|x) = tpos/(tpos+tneg)$ and $p(neg|x)=tneg/(tpos+tneg)$

Naïve Bayes:

- Assume independence between all features, conditional on the class
- Often used with categoric features

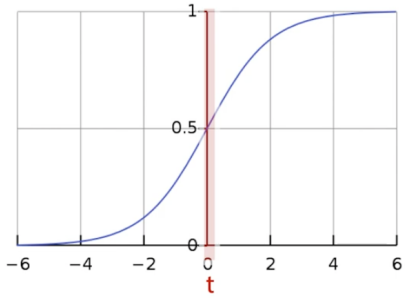
Laplace smoothing: add pseudo-observations to avoid zero probabilities.

Probabilistic classifiers: these classifiers return not just a class for a given instance x (or ranking) but a probability over all classes. This can be useful to extract a ranking (and plot an ROC curve) or we can use the probabilities to assess how certain the classifier is about its judgement. → a probabilistic classifier is also immediately a ranking classifier.

Logistic regression (is a classification model, not a regression model):

Use the sigmoid function to turn a linear classifier into a discriminative probabilistic classifier. Use log loss → maximize the log-likelihood of the data given the model. Derive the gradient and search for good weights → no analytical solutions, but the problem is convex.

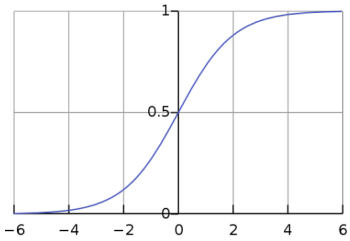
the logistic sigmoid



$$\sigma(t) = \frac{1}{1+e^{-t}} = \frac{e^{t}}{1+e^{t}}$$

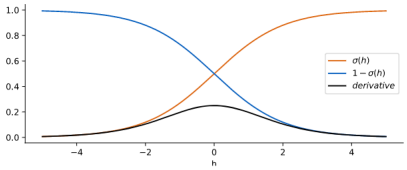
A sigmoid function is a function that makes the s-shape as can be seen above. Its domain is the entire real number line and the range is between two finite values. Benefits of the sigmoid in machine learning:

- Symmetry
- Derivative: the derivative of a sigmoid has a particularly simple form: it is equal to the sigmoid itself times one of the flipped sigmoids



symmetry

$$1 - \sigma(t) = \sigma(-t)$$



derivative

$$\sigma'(t) = \sigma(t)\sigma(-t)$$
$$= \sigma(t)(1 - \sigma(t))$$

Minimum description length principle: a model that allows us to compress the data is a model that has learned something about the data. The better the compression, the more we've learned. Balance model complexity by storing the model, and then the data given the model.

Entropy: the expected number of bits we require to compute our outcome = the codelength of each outcome, multiplied by its probability, summed over all outcomes. The entropy says something about how much uncertainty there is about the outcome, in one single value.

P(X): source of our data
Q(X): our model
Cross entropy: expected codelength if we use q, but the data comes from p.
Cross entropy is minimal when p=q (only the case if the model is perfect).
Log-loss is also known as cross-entropy loss.

Machine Learning, Lecture 5: Data pre-processing

How to clean up your data?

- Missing data
  - Often data is incomplete:
    - Missing values:

income	status	unemployed
32000	married	true
	single	false
89000		true
34000	divorced	false
54000	married	true
		false

    - Missing labels:

income	status	unemployed
32000	married	true
2000	single	
89000	single	true
34000	divorced	false
54000	married	
34000	married	false

We need to do something about these gaps before we can use a machine learning algorithm on this data. Depending on whether values from the feature columns are missing, or values from the target column are missing, we should take another approach.

  - If we have missing values in one of our features, the simplest solution is just to remove that feature with missing values. We can also remove the instances wit missing fata. This is risky. If the data was not corrupted uniformly, removing rows with missing values will change the data distribution. It is very difficult to see if the data was corrupted uniformly, or if only 1 subgroup caused gaps in the dataset.
  - Imputation: at some point we might need to fill in the missing values. We call this imputation. For categorical data we usually use the mode (= most common category), and for numerical data we usually use the mean/median. A more involved bur also more powerful way is to predict the missing value from the other features. (you can turn the feature column into a target column and train a classifier for categoric features, and a regression model for numeric features.)

    - When looking at the test set we cannot use imputation or ignore the missing values as this changes the task in a way that will give us false confidence. Instead, we should report the uncertainty created by the missing values.
- Unnatural outliers: outliers caused by, for example, broken measurement instruments. When easily recognizable, these outliers can simply be removed.
- Natural outliers: recognize that your data is not normally distributed and adapt your model to it, for instance by removing assumptions of normally distributed data.
- Class imbalance: use a bigger test set than you would normally do as the problem essentially is the detection of instances of the minority class. This is usually a difficult step, as by expanding the test data the training data often becomes very limited. The most common approach is to oversample the minority class by sampling with replacements. The advantage is that this leads to more data. However, it does result in duplicates in our dataset. This could increase the risk on overfitting. Under sampling is also an option. This does not lead to duplicates, but it does mean we are throwing away a lot of data. A more sophisticated approach is to oversample the minority data with new data derived from existing data.

How to determine the features you want to use from your data:
- Determine if you have/need numerical or categorical features, or maybe both.
- Sometimes, organizing numerical data into bins can result in a usable categorical organization of data. The other way around, if you only have categorical data and you number the categories, you will end up with perfectly usable numerical data.

Data normalization:
- Normalization: reshapes all values to lie within the range [0, 1]
- Standardization: reshapes the data so that its mean and variance are those of a standard normal distribution. (0 and 1 respectively)
- Whitening: looks at features together, to make sure that as a whole their statistics are those of a multivariate standard normal distribution.
- Normalization and standardization are usually good enough. Whitening is more powerful, but usually not necessary.

Principal component analysis (PCA)
- When we are dealing with too many features, we can select a subset of the existing feature to operate on = feature selection.
- We can also map the features to a new smaller set of features. This way we can retain as much information as possible = dimensionality reduction.
- Dimensionality reduction is the opposite of feature expansion, which we have seen earlier. It describes methods to reduce the number of features in our data by deriving new features from the old ones, hopefully in such a way that we capture all the essential information. There are several reasons you might want to reduce the dimensionality of your data:
    - Efficiency: some machine learning models can handle a limited amount of features. Therefore, sometimes reduction is necessary in order to be able to run the chosen model.
    - Reduction of variance: feature expansion boosts the power of your model, likely giving it higher variance and lower bias. Dimensionality reduction does

the opposite: it reduces the power of your model likely giving you higher bias and lower variance.
    - Visualization: sometimes you are able to reduce your data to 2 or 3 dimensions while still preserving the important information → this makes a data visualization much easier.
- Dimensionality reduction makes most sense when there are correlations between your features.

- PCA: dimensionality reduction, using linear transformations. Principal components are a natural basis for your data.

- Feature expansion: more expressive models, lower bias, higher variance

- Dimensionality reduction: reduce model expressivity, higher bias, lower variance

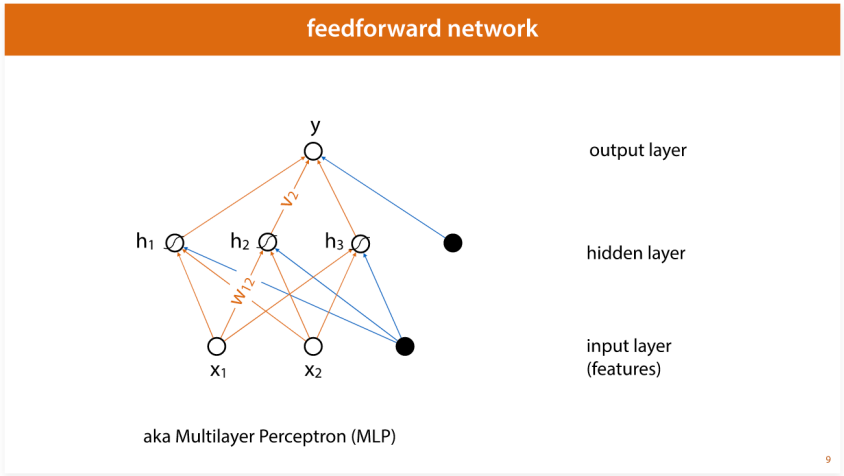
Machine Learning, Lecture 6: Beyond Linear Models
Neural network: learns a feature extractor together with the classifier
SVMs (support vector machines): use a kernel to massively expand the feature space

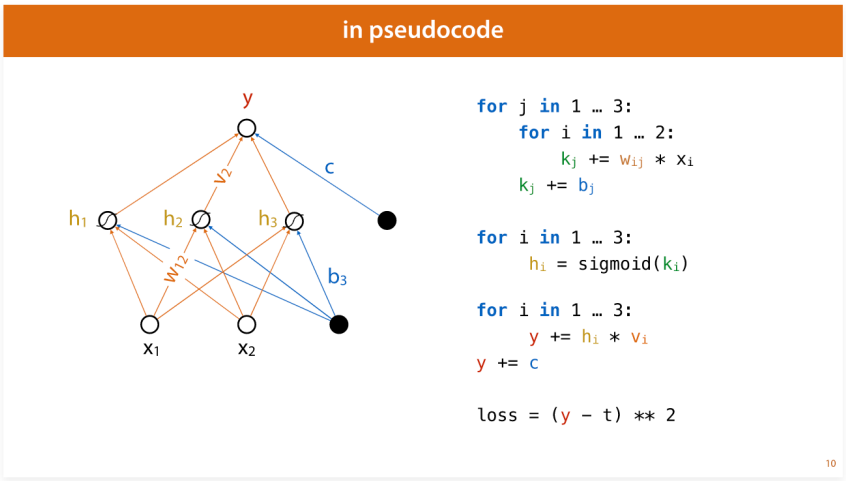
Using SVM is becoming a lot less popular → need to understand the general idea, but is not important for the exam.

Basics of neural networks:
Perceptron: basic principle = multiple inputs, one output. The perceptron has a number of inputs, the features in modern parlance, each of which is multiplied by a weight. The result is summed, together with s bias parameter, and the sign of this result is used as the classification. The bias in a perceptron can be represented as just another input that we fix to always be 1 (we call this a bias node). We can chain an infinite amount of perceptrons together, the result will always be a simple linear function over our inputs: a single perceptron.

To create perceptrons that can be chained together in such a way that the result will be more expressive than any single perceptron could be, we can include a non-linearity → an activation function. Not using an activation function is also called using a linear activation.

Using these nonlinearities, we can arrange single perceptrons into neural networks. For a long time, the most popular neural network was the feedforward network or multilayer perceptron (MLP). We arrange a layer of hidden units in the middle, each of which acts as a perceptron with a nonlinearity, connecting to all input nodes. Then we have one or more output nodes, connecting to all hidden units. Note the following points:
- There are no cycles, the network feeds forward from input to output
- Nodes in the same layer are not connected to each other, or to any other layer than the next and the previous one
- Each layer is fully connected to the previous layer, every node in one layer connects to every node in the layer before it





To build a regression model, all we need is one output node without an activation. This means that our network as a whole, describes a function from the feature space to the real number line. The number of hidden nodes is a hyperparameter: more nodes make the network more powerful, but also more likely to overfit, more expensive to compute and potentially more difficult to train. After we have computed the output, we can apply any regression loss function we like, such as least-squares loss.

Softmax activation: we create a single output node for each class, and then ensure that they are all positive and that together they sum to one. This allows us to interpret them as class probabilities.

Since neural networks can be expensive to compute we tend to use stochastic gradient descent to train them. This is very similar to gradient descent, but the loss function is defined over a single example instead of summing over the whole dataset. Then loop over all instances, and perform a small gradient descent step for each one based on only the loss for that instance. This method has several advantages:

- Using a new instance each time adds some noise to the process, since the gradient will be slightly different for each instance, which can help to escape local minima

- Gradient descent works fine if the gradient is not perfect, but still good on average. This means that taking many small inaccurate steps is often much better than taking one very accurate big step
- Computing the loss over the whole dataset is expensive. By computing the loss over one instance at a time, we get N steps of stochastic gradient descent for the price of one step of regular gradient descent.

Summary of training a neural network:

- Get some examples of input and output
- Get a loss function
- Work out the gradient of the loss wrt the weights
- Use (stochastic) gradient descent to improve the weights bit by bit

Computing the gradients for the neural networks quickly becomes too complex to work out by hand → we need to automate this process → there are three ways to work out derivatives and gradients automatically:

- Symbolically: basically how we do it with pen and paper, but then letting eg Wolphram Alpha do it for us → as we build bigger and bigger networks, the expression of the gradient will grow too big to store in memory
- Estimate of the gradient: pretty unstable, expensive
- Backpropagation: middle ground, it odes part of the work symbolically, and part of the work numerically. We get a very accurate computation of the gradient, and the cost of computation is usually only twice as expensive as computing the output for one input.

Basic algorithm of backpropagation:

- Break you computation down into a chain of modules
- Work out the derivative of each module with respect to its input symbolically
- Compute the global gradient for a given input by multiplying these gradients
- Accumulate your gradients down the computation graph

The first thing we have to do is draw a proper computation graph: in a proper computation graph, all the values that go into our computation and come out of it are nodes, whether they are parameters of a model or inputs. In the computation graph, the computation of the loss should be included. This is because we compute gradients and are always interested in the derivative of the loss with respect to the parameters.

For a complex computation graph, it is very important to work out the derivatives in the right order → that way we can reuse what we have already computed:

- Do a forward pass (compute the output (loss) given the inputs)
- Start at the top (output) of your computation graph (the computation graph should always have a single output
- Compute the derivative for every node (the derivative of the output wrt the node)
- Work your way down (this makes the computation local)

General rule about backpropagation: if we have a node feeding into a computation, the global derivative of that node (the loss over the value of the node) is always the global derivative of the output of the computation, times the local derivative of the output over the input.

Full backpropagation algorithm:

- Break your function into a computation graph, call the last node the loss
- Do a forward pass
- Start at the loss node and work your way down
- At each input to a computation, multiplying the derivative of loss over output, the derivative of output over input, produces the derivative of loss over input

Beyond linear models: maximum margin loss, there are several names for this idea, but they are used in different contexts.
Maximum margin loss = hinge loss (usually refers to the constraint-free formulation of this loss. Used in combination with neural networks) = support vector machine (usually refers to the use of this loss in combination with the kernel trick) = maximum margin hyperplane classifier (old-fashioned name for the SVM. Usually refers to the version without the kernel trick)

How does maximum margin loss work: if we have two linearly separable classes, a decision boundary exists that separates the data perfectly. We are looking for the hyperplane: the plane that separates the classes and has a maximal distance to the nearest positive point and the nearest negative point. The points closest to the decision boundary are called the support vectors. The distance to the support vectors is called the margin and we assume the decision boundary is chosen in a way that the margin is the same on both sides. → How do we maximize the margin? → the hyperplane we will choose is the one that produces y = 1 for the positive support vectors and y = -1 for the negative support vectors.

Since we don't like to maximize, we use the inverse of the maximum margin loss and minimize the objective. The resulting classifier is called a hard margin support vector machine (SVM).

- SVM does not work well when the data is not linearly separable, or if we could have a very nice decision boundary if we just ignored a few misclassified points. For instance, when there is a little noise or a few outliers.

Soft margin: we allow a few points to be on the wrong side of the margin, if it helps us to achieve a better fit on the rest of the points. To achieve this, we can use slack parameters for each point. → after this we have two options:

- Express everything in terms of w, get rid of the constraints
  - This allows gradient descent to be used
  - Good for use with neural networks/deep learning
- Express everything in terms of the support vectors, get rid of w
  - Doesn't allow error to propagate back
  - Allows the kernel trick to be applied

Overview of all loss functions:

- The error: also known as zero one loss, is simply the number or proportion of misclassified examples. It's usually what we are interested in, but it doesn't give us a loss surface that is suitable for searching
- The least-squares loss we introduced as a simple illustration of the principle of a proxy loss for the error. In practice it doesn't usually work very well and is rarely used

- The log loss requires a sigmoid function to be added to the output of the linear function. It assumes that the result is the probability of the positive class applying to the instance, and it maximizes the log likelihood of the classes given the model parameters. Practically this boils down to minimizing the negative log likelihood of the correct class. This can also be derived from the cross-entropy between the true class distribution given by the data and the class distribution given by the model
- Soft margin SVM loss, which attempts to maximize the margin between the positive and negative points. It's also known as a maximum margin loss, or the hinge loss

Machine Learning, Lecture 7: Deep Learning

Definition of a neural network: a neural network is a model described by a graph; each node represents a scalar value. The value of each node is computed from the incoming edges by multiplying the weight on the edge by the value of the node it connects to. We train the model by tuning the weights. Each line between nodes represents one of these weights.

In order to scale up the principle of backpropagation on neural networks, we need to move from operations on scalars to the linear algebra view: everything is a vector, matrix, or a higher dimensional analog of that. These vectors, matrices and higher dimensional analogs are called tensors.

Tensor:

- A tensor is a generalization of vectors and matrices to higher dimensionalities. It is a collection of numbers arranged in a grid. The rank of the tensor is the number of dimensions along which the values change.
- A vector is a rank 1 tensor: a one-dimensional array which can be described by one integer
- A matrix is a rank 2 tensor: a two dimensional array which can be described by two integers; how many rows and how many columns
- The tensor is the basic data structure of numpy → it is called a multidimensional array
- Tensors can only contain numbers → any categoric features need to be converted to numeric features. This is normally done by one-hot coding

In order to build a computation graph using tensors, we'll need functions that consume tensors and spit out new tensors. A function can have multiple inputs and multiple outputs, all of them being tensors. Sometimes, non-tensor inputs are allowed, but when computing gradients only gradients over tensor inputs will be computed.

By putting tensors and functions together, we can build a model. We define some functions and we them chain them together into a computation graph.
A deep learning system uses a computation graph to execute a given computation (the forward pass) and then uses the backpropagation algorithm to compute the gradients for the data nodes with respect to the output (the backward pass).

Automatic differentiation: perform computation by chaining functions → keep track of all computation in a computation graph → when the computation is finished, walk backward through the computation graph to perform backpropagation.
Two approaches to automatic differentiation:

- Lazy: you define your computation graph, don't place any data in it → compile the computation graph and start feeding data through it.
  + Fast, many possibilities for optimization, easy to serialize models.
  - If something goes wrong, it's very difficult to trace the program error.
- Eager: you use programming statements to compute the forward pass. The deep learning system ensures that your matrices are special objects that keep track of the whole computation, so that when time comes to do the backpropagation pass, we know how to go back through the computation we performed. Eager execution is becoming the default approach.
  + Easy to debug, problems in the model occur as the model is executing, flexible: the model can be entirely different for each forward pass.
  - More difficult to optimize and serialize.
  In eager mode, we create a node in our computation graph by specifying what data it should contain. The result is a tensor object that stores both the data and the gradient over that data.

Basic rules of tensor backpropagation:

- Functions can have any number of inputs and outputs
- Inputs and outputs can be tensors of any rank
- The final output MUST be a scalar

The computation graph is always the model + the computation of the loss → that way, the computation we're using for backpropagation always has a single scalar output.

How to perform backpropagation if the result depends on a variable along different computation paths → multivariate chain rule: to work out the derivative of a function with multiple inputs we just take a single derivative for each input, treating the others as constants, and sum them.

Backpropagation with tensors: expressing the forward pass as matric multiplications may help to make things more efficient, but that doesn't workif the backwards pass still consists of a load of loops over individual scalars. The backward pass should also be expressed in a series of matrix multiplications. The fact that we assume our function always has a scalar output saves us. It means that whatever we do, the only derivatives we ever want to end up with are those of the loss with respect to some tensor in our computation graph → the gradient of any tensor T will always have the same shape as T.

Working out derivatives for high rank tensors:

- Describe the problem in terms of scalar derivatives
- Apply the scalar (multivariate) chain rule
- Rewrite the scalar computation as tensor operations

Short summary:

- Model: the application of functions to tensors

- Each function defines a backward() function:
  Given the gradient over the outputs, compute the gradient over the inputs
- Backpropagation walks backwards through the graph, accumulating the gradient product.

Working out a backward function is not usually necessary in practice: deep learning frameworks provide a large number of pre-built functions that you can chain together to do almost anything. Only when you write your own function, you need to implement the backward and forward yourself.
How to work out the backward function:

- Phrase the problem in terms of scalar derivatives: work out the derivative for one element of the input
- Use the multivariate chain rule: sum over all elements of the output variable
- Work out the general solution in terms of matrix operations

Convolutions:

- The output layer has the shape of another image, with more channels
- Output nodes are only wired to nearby nodes in the previous layer
- Weights are shared, each hidden node has the same weights to the previous layer
- Max-pooling reduces the image dimensions

How to make deep neural nets work? → there are four most important tricks that we use to train neural networks that are big and deep (many parameters and many layers). These tricks are also the main features of any deep learning system:

- Overcoming vanishing gradients
- Minibatch gradient descent
- Optimizers
- Regularizers

Overcoming vanishing gradients: There are two standard initialization mechanisms, the idea of both is that we assume that the layer input is roughly distributed so that its mean is 0 and the variance is 1 in every direction (we must standardize or normalize the data, so this is true for the first layer). The initialization is then designed to pick a random matrix that keeps these properties true.

Minibatch gradient descent: like stochastic gradient descent, but with small batches of instances, instead of single instances. Smaller batches: more like stochastic GD, more noisy, less parallelism. Bigger batches: more like regular GD, more parallelism, limit is (GPU) memory. General advice: keep it between 16 and 128 instances.

Optimizers: attempt to adapt the gradient descent update rule to improve convergence → examples: momentum and nesterov momentum.

Regularizers: the bigger your model, the greater the capacity for overfitting. Regularizers attempt to pull the model back towards more simple models, without entirely removing the more complex ones. Different regularizers:

- L2: simpler means smaller parameters

- L1: simpler means smaller parameters and more zero parameters
- Dropout: randomly disable hidden units. Simpler means more robust

Deep learning allows us to make each module differentiable, it ensures we can work out a local gradient so that we can also train the pipeline as a whole using backpropagation → we call this end-to-end learning.

Machine Learning, Lecture 11: Sequences

Before we can look into how we can model sequences, we need to look at how we can interpret sequential data.

- Numeric 1-dimensional data: features can be divided into numeric and discrete
- Numeric n-dimensional data: sequential data can also be multidimensional
- Symbolic (categorical) 1-dimensional data: if the elements of our data are discrete, it becomes a sequence of symbols. Language is a prime example. We can model language as a sequence in two different ways: as a sequence of words, or a sequence of characters
- Symbolic n-dimensional data: we can also encounter sequences with multiple categorical features per timestamp. The more complex the sequence grows, the more difficult it can be to represent.

Sequential data:

- Sequences consisting of numbers, vectors or symbols
- Dataset: consisting of a sequence per instance, or a sequence of instances
- Both can be converted to fit classic machine learning through feature extraction

Markov models:

- Probabilistic model for sequences
- Similar to Naïve Bayes
- Estimates probabilities of small subsequences from relative frequencies in the data

Modeling sequences: we want to model the probability of a sequence occurring. When modelling probability, we usually break the sequence up into its tokens and model each as a random variable. These random variables are not independent.

Estimating probabilities: break your sequence up int subsequences, estimate their probability and combine the probabilities of the subsequences, to get the probability of the whole sequence.

Chain rule of probability (has nothing to do with the chain rule of calculus): allows us to break a joint distribution on many variables into a product of conditional distributions. This tells us that if we build a model that can estimate the probability $p(x|y|z)$ of a word x based on the words y, z that precede it, we can then chain this estimator to give us the joint probability of the whole sentence x, y, z.
A problem with many sentences of a certain length, is that they have never been seen before in their entirety. A simple way to get a basic model is to limit how far back we look in the sentence. This is called a Markov assumption.

Markov assumption: take the probability of a word given all the words that precede it and assume that it's equal to the probability of the word conditioned only on the k words that precede it. (we know this is not correct, but it does yield a very usable model)

Using the Markov assumption and the chain rule together, we can model a sequence as limited-memory conditional probabilities. These probabilities can then be very simply estimated from a large dataset of text, called a corpus. We call this a Markov model. The size of the memory used in the Markov model is the order of the Markov model.

Sequential sampling is also known as autoregressive sampling. In the context of Markov models, the sampling process is often called a Markov chain.

Deep learning on sequences:

- Sequence models operate on inputs of different lengths
- Input: raw sequence data. When we want to do deep learning, the input should be represented as a tensor
- Layers: sequence-to-sequence layers
- Output: sequence (next token prediction, translation), single vector (classification, regression)

Embedding vectors:
Assign each input symbol in your vocabulary a vector of random values. Then translate a symbolic input sequence into a sequence of vectors by mapping the input symbol to their corresponding embedding vectors. The dimensionality of the embedding vectors is a hyperparameter. The fundamental trick of embedding vectors is that we treat these vectors as parameters of the model. We feed this input sequence to the model, compute the loss, backpropagate, and get gradients on all parameters of the model, including these embedding vectors. Embedding vectors can also be pre-trained.

Sequence-to-sequence layers: we need layers that we can stack on top of each other. These need to be sequence-to-sequence layers. These are layers that take a sequence of vectors as input and produce a new sequence of vectors as output.
The defining property of a sequence-to-sequence layer is that they can consume sequences of different lengths with the same set of weights.

Different model configurations:

- Sequence-to-sequence: dataset consists of an input set and target sequences. Model is simply created by stacking a bunch of sequence-to-sequence layers and the loss is the difference between the target sequence and the output sequence.
- Sequence-to-label: we need to construct a model that consumes sequences of variable lengths, but at some stages reduces them down to a single label.
- Label-to-sequence: take the input vector and repeat it into a sequence of the same vector.

Recurrent neural networks and LSTMs:
One of the most popular neural networks for sequences is the recurrent neural network. This is a generic name for any neural network with cycles in it.

Recurrent neural network = RNN. How to train RNNs → provide an input sequence x and a target sequence t → backpropagation through time (BPTT)

- Sequence-to-sequence layer
- No Markov assumption: potentially infinite memory
- In practice: quite limited memory

LSTMs (Long short-term memory):

- Selective forgetting and remembering, controlled by learnable "gates"
- Possibly the first successful deep neural network

Short summary:

- Sequence modeling: we can use existing methods through feature extraction, but be careful about validation
- Markov models: simple, finite-memory sequence modeling
- Word2Vec: word embeddings reflecting similarity, semantics
- LSTMs: incredibly powerful language models. Tricky to train, very opaque

Machine Learning, Lecture 10: Trees and Ensembles

- Decision trees are not a very popular model in machine learning: they are quick to train, but also prone to overfitting, and regularizing them hurts the performance significantly. Decision trees are mainly used in ensembles, a model that combines a lot of other models in order to arrive at a prediction.
- Decision trees in their simplest form work on data with categorical features. Standard algorithm:
    - Start with an empty tree
    - Extend step by step, adding internal nodes
    - Greedy (no backtracking)
    - Choose the split that creates the least uniform distribution over the class labels in the resulting segmentation
    - Stop conditions:
        - When maximum depth (optional hyperparameter) has been reached
        - When all feature values are the same
        - When all class values are the same
- The best feature to split on in a tree, is the feature that creates the most non-uniform class distribution in the resulting segment. We can use entropy to establish how uneven a class distribution is, and the more uneven, the better we like the split. But to evaluate the split, we need to look at all distributions after the split and the one before.
- Conditional entropy: entropy of a conditional distribution, summed over all values of the conditional, weighted by the marginal probability of that value.
- Information gain: measures how much knowing the value of x will decrease the entropy of z.
- The larger a decision tree grows, the more likely it is to overfit on the data. (meaning training accuracy increases, but test accuracy decreases) To reduce overfitting, we can prune a tree. This means we work backwards from the leaves, with data we have withheld from training and testing (this is also not the validation set, it has to be new,

withheld data!!!). For each leaf we check whether the tree classifier is better with or without the leaf. If it works better without, we remove the node. We continue pruning until the performance stops to increase.
- We have seen classification trees that use numerical features, but if the target label is numerical, we speak of a regression tree.
- Combination of tree models = regression forest model.

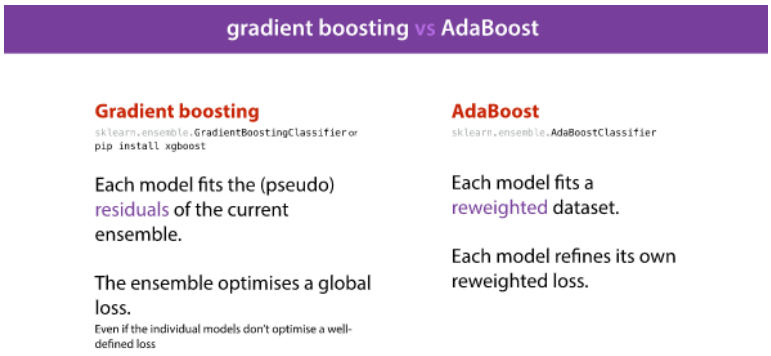
Ensembling: combining multiple models into one.
- We can simply combine the predictions
- Or we can treat predictions from separate models as features themselves, we call this stacking. We then train a new model, a combiner, on this new set of features → the combiner is a simple, low-variance model like a logistic regression.

Bootstrapping: simulating the sampling of multiple datasets from the source of our original data. → Bootstrap aggregating = bagging = resample k datasets and train k models, this ensemble classifies the majority vote.

Random forest: simple instantiation of bagging with decision trees. A bootstrapped ensemble of decision trees is trained and for each we subsample both the instances and the features we include.

Ensembling and bagging:
- Used in production to achieve extra performance on top of a particular model
- Rarely used in research
- Can be expensive for big models
- Bagging reduces variance
- Bagging helps models with a high variance and low bias (meaning: models that tend to overfit).

What if we have an underfitting model? → boosting!
Most boosting methods work by adding weight to each instance in the data. For each new model, this weight is lowered for the points the previous models got right and increased for the points the previous models got wrong. We then train the next model to focus on the reweighted version of the data.
- Gradient boosting = boosting for regression models. Main intuition: fit the next model to the residuals of the current ensemble.



Machine Learning, Lecture 8: Density estimation

Normal distribution:

- Very popular because it has a definite scale
- σ = standard deviation. Standard deviation squared = variance
- The formula for the normal distribution says that the probability that we measure the value x, depends primarily on the distance between the true value μ and x, our "measurement error". The likelihood of seeing x scales with squared exponential decay. The variance functions to scale the range of likely values.

Maximum likelihood:

- The goal of maximum likelihood is to find the optimal way to fit a distribution to the data.
- Sometimes we have a weighted dataset (e.g. in case we trust certain measurements more than others, we can downweigh some of the less trusted measurements)

K-means algorithm:

- Two unknowns are where the centers of the clusters are, and which cluster each point belongs to. If we knew which cluster each point belonged to, it would be easy to work out the center of each cluster. If we knew the cluster centers, it would be easy to work out which cluster each point belongs to → since we know neither, set one of them to an arbitrary value and then assign the points to the obvious clusters → then fix the cluster memberships and recompute cluster centers → repeat this until a good solution is converged.

Hidden variable model:

- The data is produced by picking a component, and then sampling a point from the component, but we don't see which component was used → we'll indicate which component we've picked by a variable z. This is a discrete variable, which for a three-component model can take the values 1, 2, or 3. The problem is that when we see the data, we don't know z. All we see is the sampled data, but not which component it came from → we call z a hidden variable.

Expectation-maximization:

- Informal statement of the EM algorithm:
  - Initialize components randomly
  - Loop:
    - Expectation: assign soft responsibilities to each point
    - Maximization: fit the components to the data, weighted by responsibility
- Why do we use EM:
  - Fraud detection, targeting treatment, customer segmentation
    - All unsupervised methods, mostly useful for exploratory analysis
  - EM can be used as a clustering algorithm, works similar to k-means, only we get cluster probabilities instead of cluster assignments.
  - If the EM model fits well, we can also use it for density estimation
- Formal overview of EM:
  - Allows us to prove that EM converges (to a local optimum)

    - Shows that the sample mean/variance weighted by the responsibilities are correct solutions
    - Provides a decomposition that we can reuse for other hidden variable models
- EM converges:
    - Expectation: keeps likelihood the same
    - Maximization: increases likelihood

    → Thus, we always converge to a local maximum in the likelihood

Social impact of EM:

- Profiling: e.g. when we suspect people of a crime or target them for investigation, based on their membership of a group rather than based on their individual actions. This also often leads to a sampling bias as data will not be a fair representation of the population → bias is amplified by ML models.

Machine Learning, Lecture 9: Deep Generative Models

- Mode collapse: there are several acceptable answers, but instead of picking one of the acceptable options it averages the, resulting in 1 terrible answer.
- Generator network: Put probability distribution at the start of the network, sample a point from a straightforward distribution and feed it to a neural net → the result of these steps is a random point → we have defined another probability distribution.

Training a generator:

- Loop:
    - Sample a random instance x from the data
    - Generate a random output y
    - Compute loss (x,y) and backpropagate
    - → mode collapse → does not work
- Generative Adversarial Networks (GANs): train an adversary to tell fake data from real data
- (Variational) Autoencoders: train an encoder to tell us which data point a generated point should be compared to

GANs:
How to train a GAN:

- Train a classifier to tell Xpos from Xneg
- Generate adversarial examples (clearly not Pos, but the classifier thinks so anyway)
- Add the adversarial examples to Xneg
- The classifier (discriminator) gets more robust, the generator gets more realistic

(we can think of this training as a kind of iterated 2 player game: the discriminator (classifier) tries to get good enough to tell fake data from real data and the generator (the algorithm that generates the adversarial examples) tries to get good enough to fool the discriminator. = the basic idea of the generative adversarial network

Generating adversarial examples by gradient descent is possible, but it is much nicer if both out generator and our discriminator are separate neural networks. This will lead to a much cleaner approach for training GANs.

Different types of Gans:

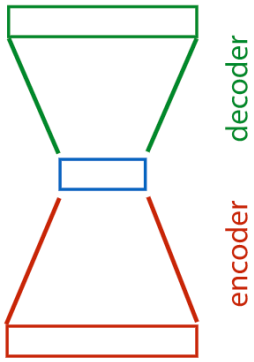
- Vanilla GAN:
    - The generator takes an input sampled from a standard MVN and produces an image. We don't give it an output distribution.
    - The discriminator takes an image and classifies it as positive (a real image of the target class) or negative (a fake image sampled from the generator).
    - If we have any other images that are not of the target class, we can add those to the negative examples as well, but often, the positive class is just our dataset, and the negative class is just fake images created by the generator.
    - To train the discriminator, we feed it examples from the positive class and train it to classify these as Pos. We also sample images from the generator and train the discriminator to recognize these as negative.
    - To train the generator, we freeze the discriminator and train the weights of the generator to produce images that cause the discriminator to label them as positive.
- Conditional GAN:
    - If we want to train the network to map an input to an output, but to generate the output probabilistically. We want to avoid mode collapse. A conditional GAN lets us train generator networks that can do this.
    - In a conditional GAN, the generator is a function with an image input, which maps it to an image output. However, it uses randomness to imagine specific details in the output.
    - To train a conditional GAN, we give the discriminator pairs of inputs and outputs. If these come from the generator, they should be classified as fake (negative) and if they come from the data, they should be classified as real (positive).
    - The generator is trained in two ways:
        - We freeze the weights of the discriminator, as before, and then train the generator to produce thins that the discriminator will think are real
        - We feed it input from the data and backpropagate on the corresponding output
    - The conditional GAN does have 1 problem: it only works well if we have an example of a specific output with a specific input. For some tasks, we don't have paired images, in that case mode collapse will occur. (for example: horses and zebras are very similar and a horse picture can be turned into a zebra picture, but we don't have paired images of horses and corresponding zebra images)
- CycleGAN:
    - Cycle GAN can solve the problem of not having two paired images using two tricks:
        - We train generators to perform the transformation in both directions (zebra-to-horse and horse-to-zebra). Then each horse in our dataset is transformed into a zebra and back again. This gives us a fake zebra picture, which we can use to train a zebra discriminator, together with real zebra pictures. We do the same the other way around.

        - Second, we add a cycle consistency loss. When we transform a horse to a zebra and back again, we should end up with the same horse again. The more different the final horse picture is from the original, the more we punish the generator networks.
- StyleGAN:
    - Basically vanillaGAN, with most of the special tricks in the way the generator is constructed. It uses many tricks, but the main aspect is the idea that the latent vector is fed to the network at each stage of its forward pass.
- There are other types of GANs, but we will not go into detail about those.

Interpolation: if we take two points in the input space, and draw a line between them, we can pick evenly spaced points on that line and decode them. If the generator is good, this should give us a smooth transition from one point to the other, and each point should result in a convincing example of our output domain. We refer to the input of a generator network as its latent space.

Autoencoders:

- A simple autoencoder is a particular type of neural network, shaped like an hourglass. Its job is to make the output as close to the input as possible, but somewhere in the network a small layer functions as a bottleneck. After the network is trained, this small layer becomes a compressed low-dimensional representation of the input.
- We feed the encoder an instance from a dataset, and all it has to do is reproduce that instance in its output. We can use any loss that compares the output to the original input, and produces a lower loss, the more similar they are. Then, we just backpropagate the loss and train by gradient descent.
  The blue layer is called the latent representation of the input. If we train an autoencoder with just two nodes on the latent representation, we can plot what latent representation each input is assigned. If the autoencoder works well, we expect to see similar images clustered together.



- What we get out of an autoencoder, depends on which part of the model we focus on:
    - If we keep the encoder and the decoder, we get a network that can help us manipulate data in this way.
    - If we just keep the encoder, we get a powerful dimensionality reduction methos.
    - If we just keep the decoder, we get a generator network.

Variational autoencoders:

- The variational autoencoder is a more principled way to train a generator network using the principles of an autoencoder.

GANs and VAEs

Allow us to train generator networks, avoiding mode collapse

GANs	VAEs
Better for images, often poor in other domains.	Work for language, music, etc.
Ad-hoc model, difficult to establish what is being optimized.	Derived from first principles.
Can't handle discrete data easily.	Allow mapping from data to latent space.
Derived by *inverting* a discriminator.	Can't handle discrete latent variables easily.
	Derived by *inverting* a generator.

VAEs and PCA

Mapping to low-dimensional *whitened* latent space (zero, mean, decorrelated).

PCA	VAEs
Linear transformation	Nonlinear transformation
Analytical solution	GD required
Principal components often meaningful, ordered by impact.	Latent dimensions not usually meaningful. *Directions* in latent space meaningful.

Machine Learning, Lecture 12: Embedding Models

- Recommender systems: a very common task for machine learning. Not quite a classification or regression, best modeled as an abstract task on its own. (Standard example: recommending movies to users)
    - In recommender systems, the primary source of data is explicit user ratings.
    - Predicting ratings based on explicit feedback is called collaborative filtering → main drawback: information is sparse.
    - Generally speaking: the abstract task of recommendation is applicable to any situation where we have two large sets of things and a particular relation between them.
- Recommending movies to users:
    - Assign a vector of initially random numbers to each user and to each movie (= the embedding vector), and we optimize the contents of these vectors to give us good representations. The number of elements k in each vector is a hyperparameter that we can set freely, but we must use the same k for both

the user and the movie embeddings. We arrange the embeddings into two matrices U and M. These are the parameters for our model.
    - In order to match user and movie, we need a scoring system to express how well movie and user match. A particularly simple way is by computing the dot product between the user embedding and the movie embedding. A second effect is the effect of magnitude. If the user is ambivalent to a certain feature, that term does not count towards the total matching score.
    - Another way of looking at the matching between users and movies is by taking the matrix of known ratings and decomposing it as the product of two factors U and M = matrix factorization.
- In movie recommendations, very often the matrix is very incomplete. We often fill the unknown ratings with zeroes → optimization only for known ratings → we define the loss for the known ratings.
- How do we work out good values for the embedding vectors? → By gradient descent (= the most versatile and scalable), another option is alternating optimization.
- The data used to recommend movies to people is almost always incomplete: people can rate movies from ambivalent to liking it very much, but they can't rate a movie with a score < 0. This makes it difficult to make an effective model → solution: negative sampling = assume some negatives based on random pairs of users and items (we can almost be certain the user won't like these movies) and treat the problem as a binary classification problem → re-use what we learned for class-labeled ratings.

Improving recommenders:

- User bias in movie recommendations is very big, some users are much more positive than others. To solve this problem, we only look at how much they deviate from their average rating of certain popular movies.
- Movie bias also exists: some movies are universally liked and others are universally loathed.
- In order to deal with model biases, we can use simple additive scalars: one for user bias, one for movie bias, and one for general bias. → This does increase the risk of overfitting.
- A big problem in recommender systems is the "cold start", meaning that when a user or a movie is new, there are no ratings available to build the embedding on. We then need to rely on implicit feedback: page views, wish lists, recorded cursor movement.
- Bias for older and newer movies is also an issue: people tend to have a more positive bias for older movies, as they are probably watching it again and already know they like it. With newer movies, people watch it out of curiosity and novelty, but it might disappoint. Therefore, older movies have a more positive bias and newer movies a more negative bias → to solve this problem, we make user embeddings time dependent.

PCA variants:

- Powerful and flexible, but:
    - Usually no analytical solution (gradient descent is always an option)
    - Sequential computation required to get ranked components (standard PCA returns ranked components analytically)

Graph models:

- Many datasets come in the form of a graph → in link prediction we assume the graph is incomplete and we try to predict the missing nodes.
- Graph convolutional networks (GCNs):
    - Depth is a problem: high connectivity diffuses information
    - Usually full-batch: no straightforward way to break up the graph into minibatches
    - Pooling is not selective: all neighbors are mixed equally before weights are applied

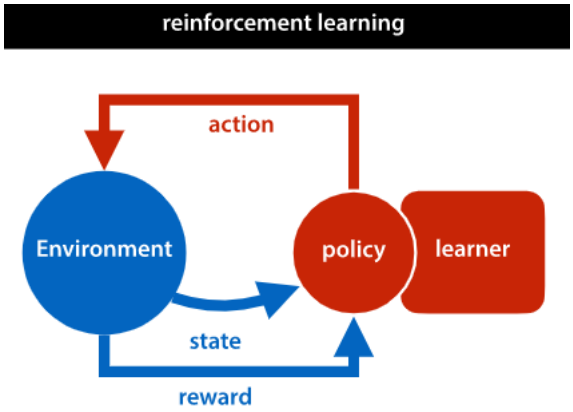
Validation of embedding models:

- Embedding models only support transductive learning (= the learning is allowed to see the features of all data, but the labels of only the training data)
- To evaluate our matrix factorization, we give the training algorithm all users, and all movies, but withhold some of the ratings

Machine Learning, Lecture 13: Reinforcement Learning

Reinforcement learning:

- Another abstract task
- = the practice of training agents that interact with a dynamic world, and to train them to learn while they're interacting



- Reinforcement learning uses the Markov decision process
- A big benefit of RL is that a single system can be developed for many different tasks, as long as the interface between the world and the learner stays the same.
- Episodic learning:
    - We define a particular activity that we'd like the robot to learn, and call that an episode. We then let the agent act one episode, based on its current policy. We observe the total reward at the end of the policy, use that to update the parameters of the policy and then start another episode.
- Problems with RL:
    - Sparse loss: very few states have a meaningful reward. Especially critical at the start of training as it will start at random.
        - Solutions:

            - Start with imitation learning (supervised learning → copying human action)
            - Reward shaping (guessing the reward for intermediate states, or states near good states)
            - Auxiliary goals (curiosity, maximum distance traveled)
        - Delayed reward: we have to decide on our immediate action, without getting immediate feedback.
        - Non-differentiable loss: some parts of the model are non-differentiable → backpropagation does not work → we estimate what backpropagation would do if the loss were differentiable.
        - Exploration vs exploitation: the more we exploit what we have learned so far, the higher the reward is in the immediate future. However, if we deviate from what we know and explore a little more, the reduction in immediate reward may be offset by the increased reward we get in the far future from understanding the environment better → you quickly get stuck in local minima. Possible solutions:
            - Boltzmann exploration: instead of sampling from the normal softmax output, we introduce a new softmax with a temperature parameter. The temperature is divided by the elements before they are exponentiated and normalized. The higher we set the temperature, the more likely we are to select those actions that we currently think are poor → this may lead to low immediate reward, but it increases the probability of finding routes to new parts of our state space that will lead to higher rewards.
            - Epsilon-greedy sampling: never sample from the probability distribution given by the policy. Instead, we just always pick the highest-probability actions. Except, with some small probability we pick a totally random action.
- Random search:
    - Very simple baseline:
        - Can be extended: simulated annealing, population methods
        - Downsides:
            - Expensive
            - Doesn't use model information
            - Sensitive to variance
- Policy gradients:
    - We can unroll the computation of an episode in our learning process
    - We cannot backpropagate through this unrolled computation graph, because parts of it are not differentiable. Specifically the sampling of actions from the output of the neural net is not differentiable, and the computation of the environment is not even accessible to us: it's essentially a secret that we are meant to discover by learning.
    - Policy gradient descent is a simple way to estimate what backpropagation would do if we could backpropagate through the whole unrolled history. We run an episode, compute the total reward at the end, and apply that as feedback for all the steps in our run. If we had a high reward at the end we

compute the gradient for a high value and follow that, and if we had a low reward we compute the gradient for a low value and follow that.
    - Train an episode, save all instances of policy network → observe total reward → distribute the total reward back to all instances in the episode → backpropagate down the network
    - Gradient estimates are unbiased, but high variance
    - Many variance reduction methods exist

Deep Q-learning:

- If we write down in a table what action we will take in every single state, we can specify a full policy. By directly learning all the values in such a table, we can actually solve a simple reinforcement learning problem: define and then find the optimal solution. This kind of analysis is often called tabular reinforcement learning (tabular Q-learning)
    - We use some policy to explore the world and we observe the consequences of taking action a in state s. Whenever we do this, we use the recursive definition of Q as an update rule: we compute on the right hand side what Q(s,a) currently is.
    - Only feasible for very small state spaces
    - No generalization between states

Tree search:

There are several simple methods of tree search:

- Random rollouts: for every node we reach by one of the moves we're considering, we simply simulate a series of random games starting at the node. This means that we just play random moves for both players until we reach a leaf node. To improve rollouts, we can use two functions:
    - Policy function $p(a|s)$: used to simulate an agent or assign probabilities of winning to a state
    - Value function V(s): used as a heuristic to value particular states during search or to estimate the expected outcome

  Rollouts with a policy and a value function: instead of playing random moves, sample moves from the policy. Limit rollout depth, and label nodes with the value from the value function.
- These policy and value functions don't need to be handwritten. They can also be learned: this is where reinforcement learning is connected to tree search.
- How to use rollouts:
    - During play: when training, we don't use tree search at all, but when the time comes to face a human player, we take our policy network, and we take our value network and we put them into the rollout algorithm. Then we play the moves that the rollout algorithm returns.
    - During training: with our starting policy, we train the next policy to mimic what the rollout algorithm does when augmented with the policy. When this learning has converged, we discard the starting policy and train the next policy → policy keeps improving.
- Minimax tree algorithm: the basic idea is that if we had sufficient compute to search the whole tree, we should be able to play perfectly → if it's possible to guarantee a

win, we should win. The idea of minimax is that the player whose turn it is labels each node with the best score they can guarantee from that node. For us, this is the maximum score, and for the opponent, this is the minimum score → that is why it is called the minimax tree.

- It is possible to include a policy function in a minimax model. This could allow us to prioritize certain nodes over others, searching them first → a policy function can help us determine which moves are more likely to yield good results. (a policy function ignores low-probability nodes (beam search) and samples the next node from the policy)
- The traditional way to use a minimax tree, is with a value function. We search the whole tree, but only up to a maximum depth. At this depth, we use the value function to label the nodes, and then work these back up the tree.

Minimax during play: use the policy and value networks to search a subtree of the gametree.
Minimax during training: use minimax as a policy/value improvement operator.

Monte Carlo Tree Search (MCTS):

- We build a subtree of the game tree in memory step by step. At first this will be just the root node, which we will extend with one child at a time. Each node, we will label with a probability: the times we've won from that node, over the total times we've visited that node. At the start this value is 0/0 for the root node, and there are no other nodes in the tree. After this, we iterate the following steps:
  - Selection: select an unexpanded node. At first, this will be the root node. But once the tree is further expanded we perform a random walk from the root down to one of the leaves.
  - Expansion: once we hit a leaf, we add one of its children to the tree, and label it with the value 0/0.
  - Simulation: from the expanded child we do a rollout
  - Backup: if we win the rollout let v = 1 otherwise v = 0. For the new child and everyone of its parents update the value. If the old value was a/b, the new value is a+v / b+1. The value is the proportion of simulated games crossing that state that we've won. This could be with a policy and a value network, or just a random rollout down a leaf.
  - From there, we proceed as before: we do a random rollout from the new node, observe whether we've won the rollout and update the values of all ancestors of the current node: we always increment the number of times visited by one, and the number of wins only if we have won the rollout.
  - Iterate
- MCTS with policies and values:
  - Policy: instead of playing random moves, sample moves from the policy
  - Value: limit the rollout depth, and label nodes with the value from the value function
- MCTS during play vs training:
  - Play: use the policy and value networks to search a subtree of the game tree
  - Training: use MCTS as a policy/value improvement operator